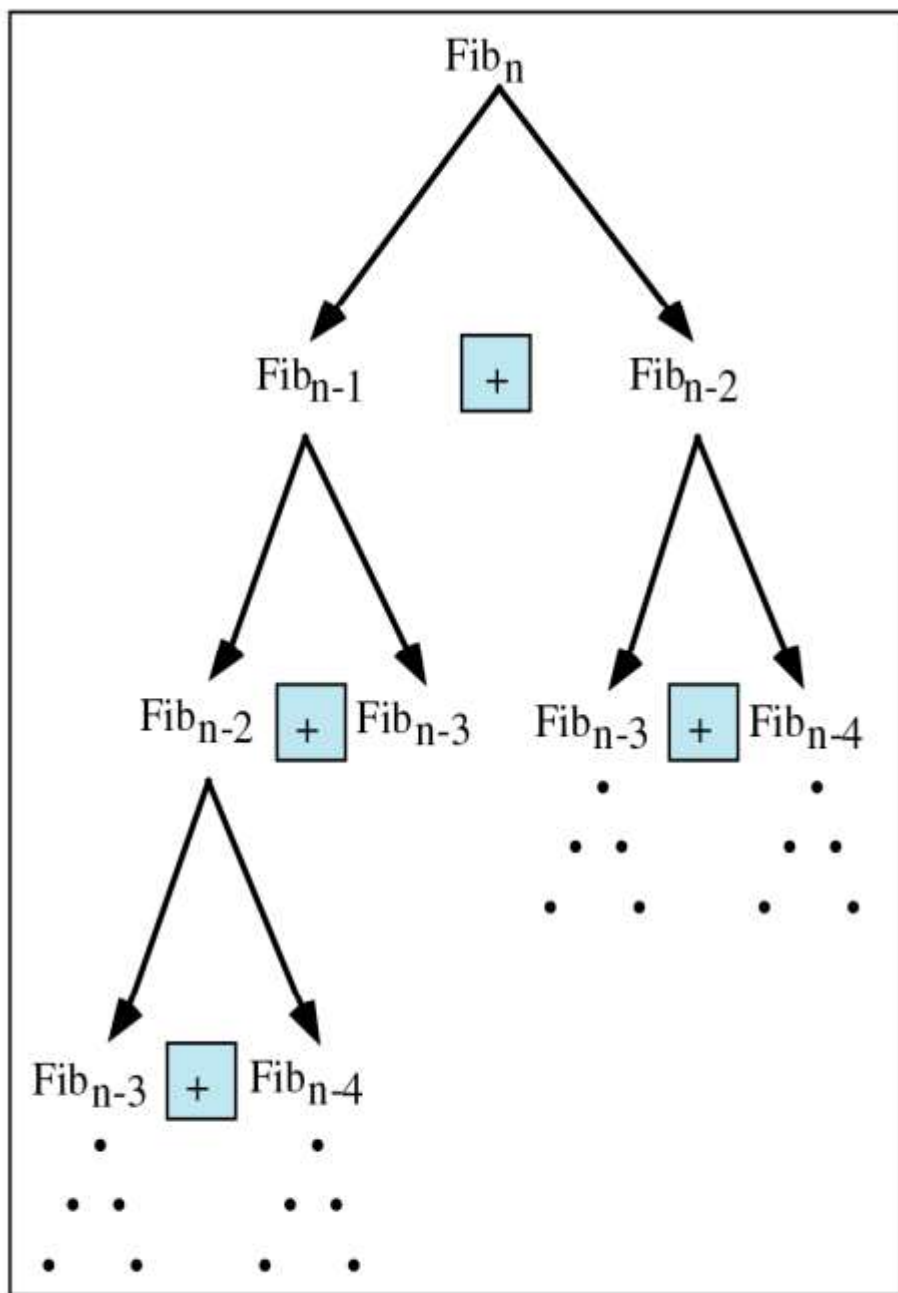


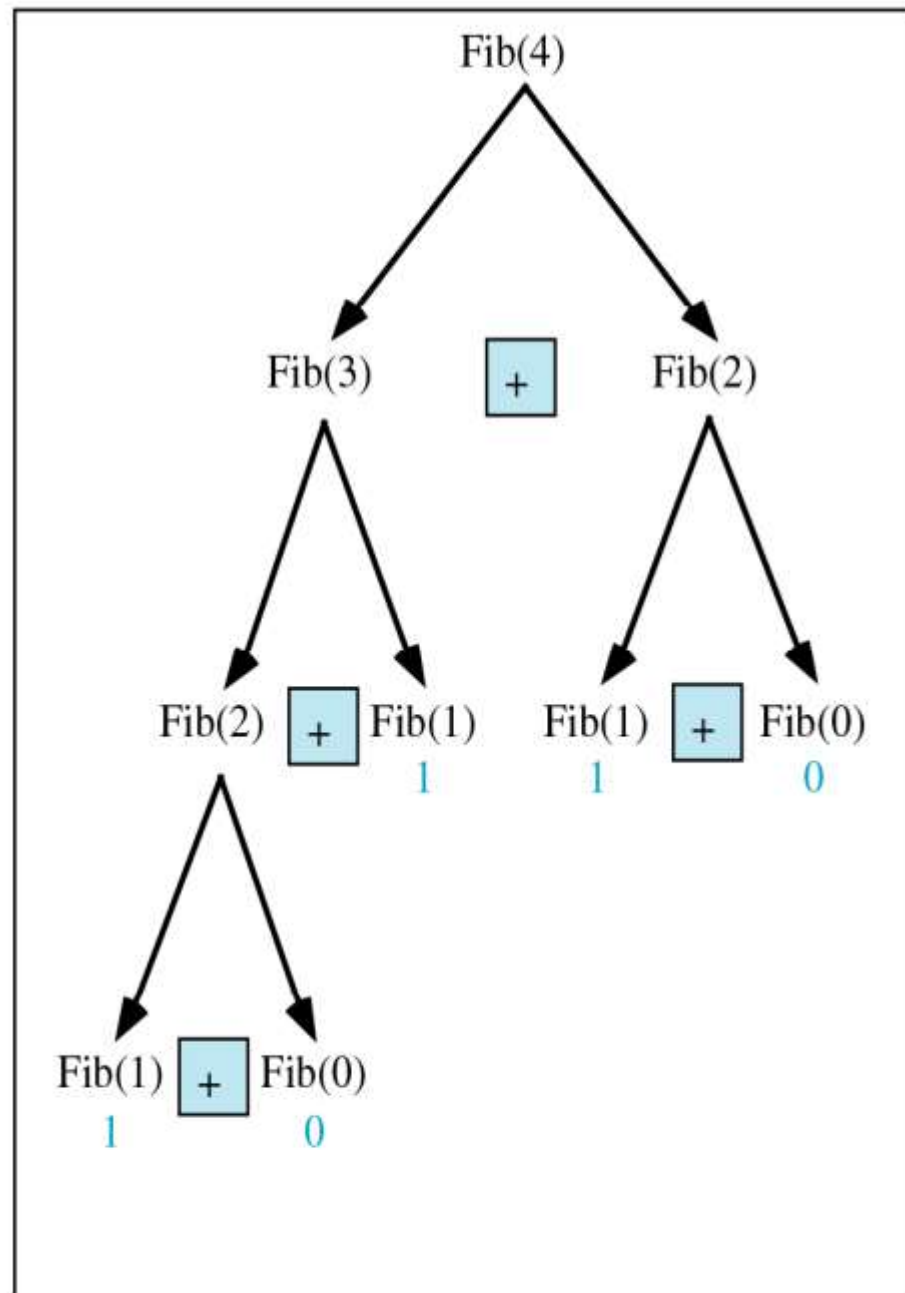
# Fibonacci Sayıları

```
int fib(int number)
{
    if (number == 0) return 0;
    if (number == 1) return 1;
    return (fib(number-1) + fib(number-2));
}
```

```
int main(){           // driver function
    int inp_number;
    cout << "Please enter an integer: ";
    cin >> inp_number;
    cout << "The Fibonacci number for "<< inp_number
         << " is "<< fib(inp_number)<<endl;
    return 0;
}
```



(a)  $\text{Fib}(n)$



(b)  $\text{Fib}(4)$

# Fibonacci Sayıları Süreç

- num=4:

fib(4):

4 == 0 ? No; 4 == 1? No.

fib(4) = fib(3) + fib(2)

fib(3):

3 == 0 ? No; 3 == 1? No.

fib(3) = fib(2) + fib(1)

fib(2):

2 == 0? No; 2==1? No.

fib(2) = fib(1) + fib(0)

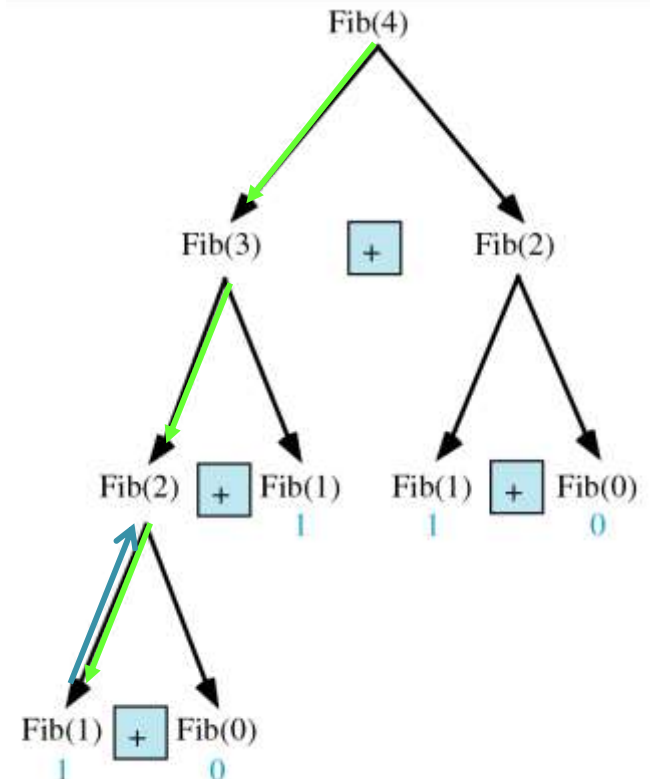
fib(1):

1 == 0 ? No; 1 == 1? Yes.

fib(1) = 1;

return fib(1);

```
int fib(int num)
{
    if (num == 0) return 0;
    if (num == 1) return 1;
    return
        (fib(num-1)+fib(num-2));
}
```



# Fibonacci Sayıları İşletim süreci

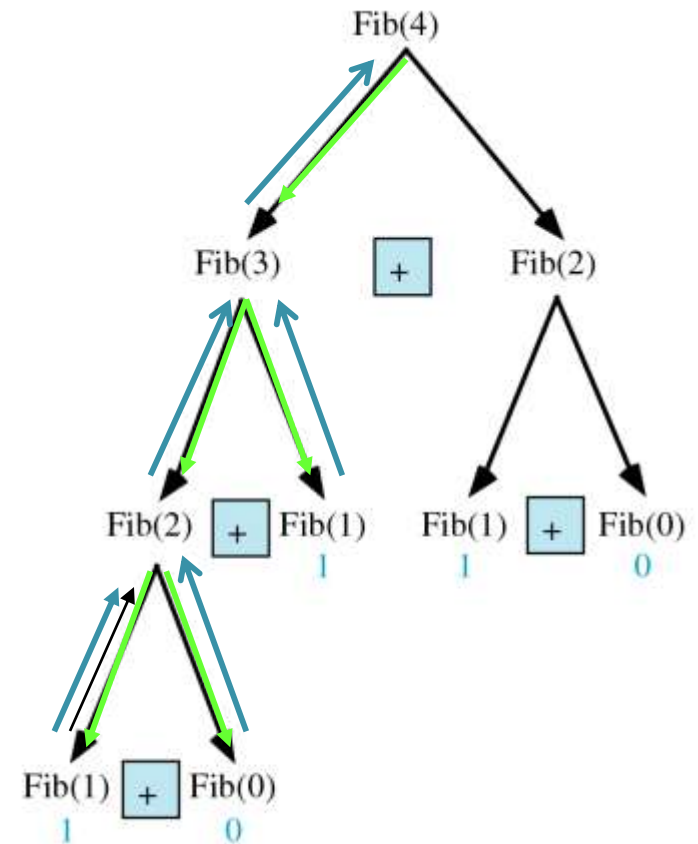
```
fib(0):  
    0 == 0 ? Yes.  
    fib(0) = 0;  
    return fib(0);
```

```
fib(2) = 1 + 0 = 1;  
return fib(2);
```

```
fib(3) = 1 + fib(1)
```

```
fib(1):  
    1 == 0 ? No; 1 == 1? Yes  
    fib(1) = 1;  
    return fib(1);
```

```
fib(3) = 1 + 1 = 2;  
return fib(3)
```



# Fibonacci sayıları işletim süreci

```
fib(2):
```

```
2 == 0 ? No; 2 == 1?No.
```

```
fib(2) = fib(1) + fib(0)
```

```
fib(1):
```

```
1 == 0 ? No; 1 == 1? Yes.
```

```
fib(1) = 1;
```

```
return fib(1);
```

```
fib(0):
```

```
0 == 0 ? Yes.
```

```
fib(0) = 0;
```

```
return fib(0);
```

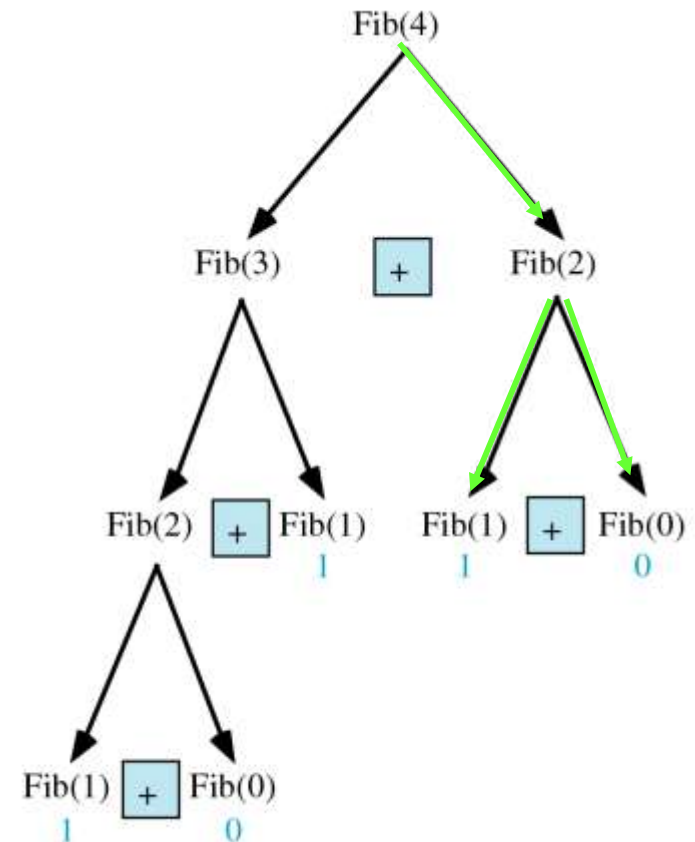
```
fib(2) = 1 + 0 = 1;
```

```
return fib(2);
```

```
fib(4) = fib(3) + fib(2)
```

```
= 2 + 1 = 3;
```

```
return fib(4);
```



# Fibonacci sayıları (özyinelemesiz)

```
int fib(int n)
{
    int f[100];
    f[0] = 0; f[1] = 1;
    for (int i=2; i<= n; i++)
        f[i] = f[i-1] + f[i-2];
    return f[n];
}
```



```
#include <cstdlib>
#include <iostream>
using namespace std;
```

```
int fib(int n){
    if (n==0) return 0;
    else if (n==1) return 1;
    else return fib(n-2)+fib(n-1);
}
```

```
int main()
{
    int n=10;
    cout<<"fib("<<n<<"")="<<fib(n);
    return 0;
}
```

# Fibonacci Numbers

- Fibonacci sayıları aşağıdaki formül ile de hesaplanabilir.

$$F_n = \frac{1}{\sqrt{5}} \left( \left( \frac{1 + \sqrt{5}}{2} \right)^n - \left( \frac{1 - \sqrt{5}}{2} \right)^n \right)$$





# Dinamik Programlama

Longest Common Subsequence (LCS)

# Dinamik Programlama

- Özyineleme yaklaşımı, bazı durumlarda oldukça zor problemlere kolay bir yaklaşım sunar.
- Bazı durumlarda ise verimsiz olabilir. Bunun sebebi belirli fonksiyon değerlerini defalarca yeniden hesaplamasından kaynaklanabilir.
  - Fibonacci örneğinde olduğu gibi

# Longest Common Subsequence Problem

- Problem: Verilen iki katar da en uzun ortak alt katarı bulmak.
- Bir katar için altkatar, dizgede görüldüğü sırada olmak üzere tüm dizgideki harflerin bir kısmıdır.
  - Örnek olarak “GOODMORNING” katarı için
    - “ODOR” *bir altkatar*dır.
      - Alt katarı oluşturan simgeler 1, 3, 5 ve 6. sıralarda oluşur.
    - “MOOD” ise bir altkatar (subsequence) değildir.
      - Çünkü simgeler sıralı olmamıştır.

# Longest Common Subsequence

- Problemin Özyinelemeli Çözümü:
  - Eğer her iki s1 ve s2 katarının son simgeleri eşleşirse,
    - $LCS = 1 + (\text{Her iki katarın son simgesinin kaldırılmış halinin LCS'si})$   
Örneğin  
“BIRD” and “FIND”
  - İki katarın ilk karakterleri eşleşmiyorsa, LCS iki seçenekten biri olacaktır:  
Örneğin:  
“BIR” and “FIN”
    - 1) Son karakteri olmayan x ve y'nin LCS'i.
    - 2) Son karakteri olmayan y ve x'in LCS'si.
  - Biz bu iki değerden max olanı kabul ederiz.  
$$\text{Max} ( LCS(\text{“BI”}, \text{“FIN”}) , LCS(\text{“BIR”}, \text{“FI”}) )$$
- (Ayrıca, x ve y'nin ilk 2 karakterini kolayca karşılaştırabilir ve benzer bir teknik kullanabiliriz. )

```
public static int lcsrec(String x, String y)
{
    // If one of the strings has 1 character, search for that
    // character in the other string and return 0 or 1.
    if (x.length() == 1)
        return find(x.charAt(0), y);
    if (y.length() == 1)
        return find(y.charAt(0), x);

    // Solve the problem recursively.
    // Check if corresponding last characters match.
    if (x.charAt(len1-1) == y.charAt(len2-1))
        return 1 + lcsrec(x.substring(0, x.length()-1),
                           y.substring(0, y.length()-1));

    // Corresponding characters do not match.
    else
        return max(lcsrec(x, y.substring(0, y.length()-1)),
                   lcsrec(x.substring(0, x.length()-1), y));
}
```

# LCS – Özyinelemeli

- $\text{LCS}(\text{"FUN"}, \text{"FIN"}) = 2$

- Son simge eşleşti:  $1 + \text{LCS}(\text{"FU"}, \text{"FI"}) = 1 + 1 = 2$

- $\text{LCS}(\text{"FU"}, \text{"FI"}) = 1$

- SonSimge Eşleşmedi:

$\max(\text{LCS}(\text{"FU"}, \text{"F"}), \text{LCS}(\text{"F"}, \text{"FI"}))$

$= \max(1, 1) = 1$

- $\text{LCS}(\text{"FU"}, \text{"F"})$

$\text{LCS}(\text{"F"}, \text{"FI"})$

TemelKısım: return 1, çünkü "F" ortak "FU" için      return 1, çünkü "F" ortak "FI" için

# Longest Common Subsequence

- Şimdi hedefimiz bu özyinelemeli çözümü almak ve dinamik bir programlama çözümü oluşturmak olacak.
  - Buradaki anahtar, her özyinelemeli çağrının kalbinin, hangi önek dizesini düşündüğümüzü söyleyen dizinler çifti olduğunu fark etmektir..
  - Bir anlamda, daha küçük LCS sorularına verilen cevaplara dayanarak "daha uzun" LCS sorularına cevap oluşturabiliriz.
- Bu son birkaç adımda özyineleme yoluyla izlenebilmektedir.

# Longest Common Subsequence

- Tekrarlayan çağrıyı RACECAR ve CREAM dizilerinde uygulayalım.
- RACECAR ve CREA girişleri ve RACECA ve CREAM girişleri için özyinelemeli çağrılara cevaplar geldiğinde
- - bu iki cevabı kullanabiliriz ve derhal sorunumuzu çözmek için bu ikisinin maksimumunu alabiliriz!



# Longest Common Subsequence

- Bu nedenle, bu özyinelemeli aramaların cevaplarını aşağıdaki gibi bir tabloda saklarız:

|   | R | A | C   | E | C | A | R |
|---|---|---|-----|---|---|---|---|
| C |   |   |     |   |   |   |   |
| R |   |   |     |   |   |   |   |
| E |   |   |     |   |   |   |   |
| A |   |   | XXX |   |   |   |   |
| M |   |   |     |   |   |   |   |

- Örneğin, bu grafikte, XXX , CREA ve RAC'ın en uzun genel alt sırasını temsil eden bir tamsayıyı depolayacaktır.

# Longest Common Subsequence

- Tabloyu oluşturmak için önce ilk satırı ve sütunu ilklendirin..
  - Temel olarak, diğer string'deki ilk harfi ararız, oraya vardığımızda 1 koyarız ve satırdan veya sütundan sonraki tüm diğer değerler de birdir.
  - Bu özyinelemeli koddaki temel duruma karşılık gelir.

|   | R | A | C | E | C | A | R |
|---|---|---|---|---|---|---|---|
| C | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| R | 1 |   |   |   |   |   |   |
| E | 1 |   |   |   |   |   |   |
| A | 1 |   |   |   |   |   |   |
| M | 1 |   |   |   |   |   |   |

# Longest Common Subsequence

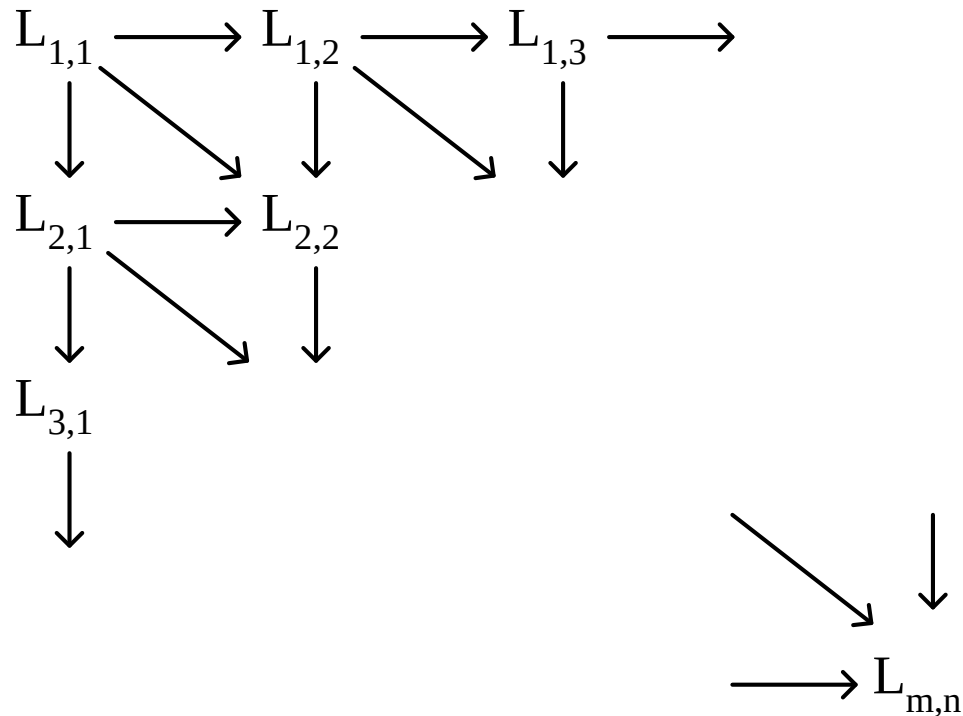
- Şimdi, özyinelemeye göre çizelgeyi dolduruyoruz:
  - 1) "Son" karakterlerin eşleşip eşleşmediğini kontrol ederiz.  
Recursive: Eğer öyleyse, bunu silin ve kalanların LCS'sini alın ve buna 1 ekleyin  
**Dinamik Programlama: Tabloda yukarıya ve sola bakın, buna 1 ekleyin.**
  - 2) Değilse, 2 olasılık deneriz ve bu 2 olasılıktan en fazlasını alırız..  
Recursive: (Bu olasılıklar basitçe tüm ilk kelimenin ve ikinci kelimenin eksi son harfinin LCS'sini alıyor)  
**Dinamik Programlama: Max (soldaki hücre, yukarıdaki hücre)**

|   | R | A | C | E | C | A | R |
|---|---|---|---|---|---|---|---|
| C | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| R | 1 | 1 | 1 | 1 | 1 | 1 | 2 |
| E | 1 | 1 | 1 | 2 | 2 | 2 | 2 |
| A | 1 | 2 | 2 | 2 | 2 | 3 | 3 |
| M | 1 | 2 | 2 | 2 | 2 | 3 | 3 |

# LCS algoritması

- $A = a_1 a_2 \dots a_m$  ve  $B = b_1 b_2 \dots b_n$
- $L_{i,j} : (a_1 a_2 \dots a_i \text{ and } b_1 b_2 \dots b_j)$  katarları için en uzun ortak altkatarın uzunluğu
- $$L_{i,j} = \begin{cases} L_{i-1,j-1} + 1 & \text{if } a_i = b_j \\ \max\{ L_{i-1,j}, L_{i,j-1} \} & \text{if } a_i \neq b_j \end{cases}$$
$$L_{0,0} = L_{0,j} = L_{i,0} = 0 \quad \text{for } 1 \leq i \leq m, 1 \leq j \leq n.$$

- LCS problemi için dinamik programlama yaklaşımı:



- Zaman Karmaşıklığı:  $O(mn)$

# LCS algoritmasında geri izleme

- e.g.  $A = b a c a d, B = a c c b a d c b$

|   |   | B |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
|   |   | a | c | c | b | a | d | c | b |   |
| A | b | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   | a | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
|   | c | 0 | 1 | 2 | 2 | 2 | 2 | 3 | 3 | 3 |
|   | a | 0 | 1 | 2 | 2 | 2 | 3 | 3 | 3 | 3 |
|   | d | 0 | 1 | 2 | 2 | 2 | 3 | 4 | 4 | 4 |
|   |   |   |   |   |   |   |   |   |   |   |

# Longest Common Subsequence

- **Dinamik Programlama Kod:**

```
public static int computeLCS(String a, String b)
{
    int[][] lengths = new int[a.length()+1][b.length()+1];

    // row 0 and column 0 are init. to 0 already
    for (int i = 1; i <= a.length(); i++)
        for (int j = 1; j <= b.length(); j++)
            if (a.charAt(i) == b.charAt(j))
                lengths[i][j] = lengths[i-1][j-1] + 1;
            else
                lengths[i][j] = Math.max(lengths[i][j-1], lengths[i-1][j]);

    return lengths[a.length()][b.length()];
}
```

---

# **0-1 Sırt Çantası Problemi**

## **(0-1 Knapsack problem)**



# Sırt Çantası problemi (Knapsack problem)

---

Verilen bazı elemanlar içerisinde,  
maksimum değeri oluşturacak kombinasyon  
seçilerek sırt çantasına konulacaktır.  
Her elemanın bir ağırlığı ve değeri vardır.  
Sırt çantasının kapasitesi  $W$  'dir.

| Eleman | $W$ (Ağırlık) | Değer( $V$ ) |
|--------|---------------|--------------|
| 1      | 1             | 8            |
| 2      | 3             | 6            |
| 3      | 5             | 5            |

# 0-1 Knapsack problem

- Problem, şöyle ifade edilebilir.

$$\max \sum_{i \in T} b_i \text{ hedefi } \sum_{i \in T} w_i \leq W \text{ sınırı ile}$$

◆ “0-1” problemi

# 0-1 Knapsack problem: brute-force yaklaşımı

---

- $n$  adet eleman olduğundan bu elemanların  $2^n$  olası eleman seçimi söz konusudur.
- Bu kombinasyonlardan maksimum değerli olan bir tanesini ağırlığı  $W$ 'den az olmak kaydıyla seçeriz.
- Running time  $O(2^n)$  olacaktır.

# 0-1 Knapsack problem:

## Dinamik programlama yaklaşımı

- Maximum  $W$  kapasiteli bir sırt çantası var ve  $n$  eleman içeren bir  $S$  kümesi
- Her  $i$  elemanı  $w_i$  ağırlıklıdır ve değeri  $b_i$  (Bütün  $w_i$  'ler ve  $W$  tam sayıdır.)
- Problem:

Sırt çantası toplam değerinin maksimum olmasını sağlayacak biçimde nasıl doldurulmalıdır?

# Alt problemi (Subproblem) tanımlamak

- Dinamik programlama ile daha az maliyetle gerçekleştirilebilir. Alt problem çok dikkatli bir biçimde tanımlanmalıdır.

Ön çalışma:

Elemanlar  $1..n$ , olarak etiketli ise alt problem

$S_k = \{1, 2, .. k \text{ etiketli elemanlar}\}$

için optimal çözümü bulmaya çalışacaktır.

Problemnin tamamının çözümü ( $S_n$ ) aceba ( $S_k$ ) ile ifade edilebilir mi?

# Alt problemin tanımı

|         |         |         |         |  |
|---------|---------|---------|---------|--|
| $w_1=2$ | $w_2=4$ | $w_3=5$ | $w_4=3$ |  |
| $b_1=3$ | $b_2=5$ | $b_3=8$ | $b_4=4$ |  |

?

Max ağırlık:  $W = 20$

**$S_4$  için:**

Toplam ağırlık: 14

Maximum değer: 20

|         |         |         |          |
|---------|---------|---------|----------|
| $w_1=2$ | $w_2=4$ | $w_3=5$ | $w_5=9$  |
| $b_1=3$ | $b_2=5$ | $b_3=8$ | $b_5=10$ |

**$S_5$  için:**

Toplam ağırlık: 20

Maximum değer: 26

| Item # | Ağırlık $w_i$ | Değer $b_i$ |
|--------|---------------|-------------|
| 1      | 2             | 3           |
| 2      | 4             | 5           |
| 3      | 5             | 8           |
| 4      | 3             | 4           |
| 5      | 9             | 10          |

$S_4$  (items 1, 2, 3, 4)  
 $S_5$  (items 1, 2, 3, 4, 5)

$S_4$  cevabı,  $S_5$  cevabının bir parçası değil.....

# Alt problem tanımlama

- Her alt problemin elemanlarının maksimum ağırlığını gösteren yeni bir parametre ekleyelim.
- Böylece alt problem sırt çantasındaki  $S_k = \{1, 2, .. k$  etiketli elemanlar} için optimal çözüm sunan  $V[k,w]$ 'nin hesaplanmasını içerecektir.

# Alt problem için Rekürsif Formül

---

- Alt problem sırt çantasındaki  $S_k = \{1, 2, \dots k \text{ etiketli elemanlar} \}$  için en iyi çözümü oluşturmak üzere  $V[k,w]$  'leri hesaplamalıdır.
- $V[i, j]$  değerlerinin bilindiği varsayımı ile, (burada  $i=0,1, 2, \dots k-1, j=0,1,2, \dots w$ )
- $V[k,w]$  nasıl hesaplanacaktır?



# Alt problem için Rekürsif Formül

---

$$V[k, w] = \begin{cases} V[k-1, w] & \text{if } w_k > w \\ \max\{V[k-1, w], V[k-1, w-w_k] + b_k\} & \text{else} \end{cases}$$

Yani Toplam ağırlığı  $w$  olan en iyi  $S_k$  alt kümesi:

- 1) Toplam ağırlığı  $\leq w$  olan en iyi  $S_{k-1}$  alt kümesi **yada**
- 2) Toplam ağırlığı  $\leq w-w_k$  olan en iyi  $S_{k-1}$  alt kümesi  
( $k$  elemanı eklenir)

# Rekürsif Formül:

---

$$V[k, w] = \begin{cases} V[k-1, w] & \text{if } w_k > w \\ \max\{V[k-1, w], V[k-1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- ◆ Toplam ağırlığı  $\leq w$  olan en iyi  $S_k$  alt kümesi elemanını içerir yada içermez.
- ◆ Durum 1:  $w_k > w$ ,  $k$  elemanı çözüme dahil edilemez. Çünkü sırt çantasına konulması halinde toplam ağırlık  $> w$  olacaktır ve bu kabul edilemez.
- ◆ Durum 2:  $w_k \leq w$ ,  $k$  elemanı çantaya **konabilir**, biz yüksek değerli durumu tercih ederiz.

# 0-1 Sırt Çantası Algoritması:

---

for  $w = 0$  to  $W$

$V[0,w] = 0$

for  $i = 1$  to  $n$

$V[i,0] = 0$

for  $i = 1$  to  $n$

for  $w = 0$  to  $W$

if  $w_i \leq w$  // Eleman çözümün bir parçası olabilir.

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

## Çalışma zamanı:

---

for w = 0 to W

$O(W)$

V[0,w] = 0

for i = 1 to n

V[i,0] = 0

for i = 1 to n

*n defa tekrar ediyor*

for w = 0 to W

$O(W)$

< kod parçasının kalan kısmı >

Bu algoritmanın çalışma zamanı?

$O(n*W)$

Brute-force yaklaşımının maliyeti  $O(2^n)$  idi.

# Örnek:

---

Aşağıdaki bilgiler için Algoritmayı işletelim:

$n = 4$  (eleman sayısı)

$W = 5$  (max ağırlık kapasite)

Elemanlar( $w$ (ağırlık),  $v$ (değer)):

(2,3), (3,4), (4,5), (5,6)

## Örnek (2)

---

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                |   |   |   |   |   |   |
| 2                |   |   |   |   |   |   |
| 3                |   |   |   |   |   |   |
| 4                |   |   |   |   |   |   |

for  $w = 0$  to  $W$   
     $V[0,w] = 0$

## Örnek (3)

---

| i\W | 0 | 1 | 2 | 3 | 4 | 5 |
|-----|---|---|---|---|---|---|
| 0   | 0 | 0 | 0 | 0 | 0 | 0 |
| 1   | 0 |   |   |   |   |   |
| 2   | 0 |   |   |   |   |   |
| 3   | 0 |   |   |   |   |   |
| 4   | 0 |   |   |   |   |   |

for  $i = 1$  to  $n$   
     $V[i,0] = 0$

## Örnek (4)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 |   |   |   |   |
| 2                | 0 |   |   |   |   |   |
| 3                | 0 |   |   |   |   |   |
| 4                | 0 |   |   |   |   |   |

$i=1$

$b_i=3$

$w_i=2$

$w=1$

$w-w_i = -1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

**else**  $V[i, w] = V[i-1, w]$  //  $w_i > w$



## Örnek (5)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 |   |   |   |
| 2                | 0 |   |   |   |   |   |
| 3                | 0 |   |   |   |   |   |
| 4                | 0 |   |   |   |   |   |

$i=1$

$b_i=3$

$w_i=2$

$w=2$

$w-w_i=0$

◆ if  $w_i \leq w$  // eleman çözümün parçası **olabilir**  
     if  $b_i + V[i-1, w-w_i] > V[i-1, w]$   
          $V[i, w] = b_i + V[i-1, w-w_i]$   
     else  
          $V[i, w] = V[i-1, w]$   
     else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

## Örnek (6)

| $i \backslash W$ | 0 | 1 | 2 | 3        | 4 | 5 |
|------------------|---|---|---|----------|---|---|
| 0                | 0 | 0 | 0 | 0        | 0 | 0 |
| 1                | 0 | 0 | 3 | <b>3</b> |   |   |
| 2                | 0 |   |   |          |   |   |
| 3                | 0 |   |   |          |   |   |
| 4                | 0 |   |   |          |   |   |

$i=1$

$b_i=3$

$w_i=2$

$w=3$

$w-w_i=1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

# Örnek (7)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| i\W | 0 | 1 | 2 | 3 | 4        | 5 |
|-----|---|---|---|---|----------|---|
| 0   | 0 | 0 | 0 | 0 | 0        | 0 |
| 1   | 0 | 0 | 3 | 3 | <b>3</b> |   |
| 2   | 0 |   |   |   |          |   |
| 3   | 0 |   |   |   |          |   |
| 4   | 0 |   |   |   |          |   |

i=1

$b_i=3$

$w_i=2$

$w=4$

$w-w_i=2$

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

## Örnek (8)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5        |
|------------------|---|---|---|---|---|----------|
| 0                | 0 | 0 | 0 | 0 | 0 | 0        |
| 1                | 0 | 0 | 3 | 3 | 3 | <b>3</b> |
| 2                | 0 |   |   |   |   |          |
| 3                | 0 |   |   |   |   |          |
| 4                | 0 |   |   |   |   |          |

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

$i=1$

$b_i=3$

$w_i=2$

$w=5$

$w-w_i=3$

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

## Örnek (9)

| $i \backslash W$ | 0 | 1        | 2 | 3 | 4 | 5 |
|------------------|---|----------|---|---|---|---|
| 0                | 0 | 0        | 0 | 0 | 0 | 0 |
| 1                | 0 | 0        | 3 | 3 | 3 | 3 |
| 2                | 0 | <b>0</b> |   |   |   |   |
| 3                | 0 |          |   |   |   |   |
| 4                | 0 |          |   |   |   |   |

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

$i=2$

$b_i=4$

$w_i=3$

$w=1$

$w-w_i = -2$

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

**else**  $V[i, w] = V[i-1, w]$  //  $w_i > w$

# Örnek (10)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| i\W | 0 | 1 | 2        | 3 | 4 | 5 |
|-----|---|---|----------|---|---|---|
| 0   | 0 | 0 | 0        | 0 | 0 | 0 |
| 1   | 0 | 0 | 3        | 3 | 3 | 3 |
| 2   | 0 | 0 | <b>3</b> |   |   |   |
| 3   | 0 |   |          |   |   |   |
| 4   | 0 |   |          |   |   |   |

i=2

$b_i=4$

$w_i=3$

$w=2$

$w-w_i = -1$

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

**else**  $V[i, w] = V[i-1, w]$  //  $w_i > w$

# Örnek (11)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 |   |   |
| 3                | 0 |   |   |   |   |   |
| 4                | 0 |   |   |   |   |   |

$i=2$

$b_i=4$

$w_i=3$

$w=3$

$w-w_i=0$

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

## Örnek (12)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 |   |
| 3                | 0 |   |   |   |   |   |
| 4                | 0 |   |   |   |   |   |

$i=2$

$b_i=4$

$w_i=3$

$w=4$

$w-w_i=1$

if  $w_i \leq w$  // eleman çözümün parçası olabilir

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$



## Örnek (13)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 |   |   |   |   |   |
| 4                | 0 |   |   |   |   |   |

$i=2$

$b_i=4$

$w_i=3$

$w=5$

$w-w_i=2$

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

## Örnek (14)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 |   |   |
| 4                | 0 |   |   |   |   |   |

$i=3$  4: (5,6)

$b_i=5$

$w_i=4$

$w=1..3$

if  $w_i \leq w$  // eleman çözümün parçası olabilir

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

## Örnek (15)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 |   |
| 4                | 0 |   |   |   |   |   |

$i=3$

$b_i=5$

$w_i=4$

$w=4$

$w - w_i=0$

if  $w_i \leq w$  // eleman çözümün parçası **olabilir**

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w - w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

# Örnek (16)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 |   |   |   |   |   |

$i=3$

$b_i=5$

$w_i=4$

$w=5$

$w - w_i=1$

if  $w_i \leq w$  // eleman çözümün parçası olabilir

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w - w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

# Örnek (17)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| i\W | 0 | 1 | 2 | 3 | 4 | 5 |
|-----|---|---|---|---|---|---|
| 0   | 0 | 0 | 0 | 0 | 0 | 0 |
| 1   | 0 | 0 | 3 | 3 | 3 | 3 |
| 2   | 0 | 0 | 3 | 4 | 4 | 7 |
| 3   | 0 | 0 | 3 | 4 | 5 | 7 |
| 4   | 0 | 0 | 3 | 4 | 5 |   |

$i=4$

$b_i=6$

$w_i=5$

$w=1..4$

if  $w_i \leq w$  // eleman çözümün parçası olabilir

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w-w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

# Örnek (18)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5              |
|------------------|---|---|---|---|---|----------------|
| 0                | 0 | 0 | 0 | 0 | 0 | 0              |
| 1                | 0 | 0 | 3 | 3 | 3 | 3              |
| 2                | 0 | 0 | 3 | 4 | 4 | 7              |
| 3                | 0 | 0 | 3 | 4 | 5 | 7              |
| 4                | 0 | 0 | 3 | 4 | 5 | $\downarrow$ 7 |

$i=4$

$b_i=6$

$w_i=5$

$w=5$

$w - w_i = 0$

if  $w_i \leq w$  // eleman çözümün parçası olabilir

if  $b_i + V[i-1, w-w_i] > V[i-1, w]$

$V[i, w] = b_i + V[i-1, w - w_i]$

else

$V[i, w] = V[i-1, w]$

else  $V[i, w] = V[i-1, w]$  //  $w_i > w$

- 
- Algoritma bu haliyle sadece çantada taşınabilecek maksimum değeri hesaplamaktadır.  $V[n,W]$  değeri
  - Bu maksimum değeri meydana getiren elemanlar nelerdir?

## Sırt Çantası içeriğini bulmak:

- İhtiyacımız olan bilgiler tabloda kayıtlıdır.
- $V[n, W]$ , sırt çantasına yerleştirebilecek elemanların maksimum değeri
- $i=n$  ve  $k=W$

if  $V[i, k] \neq V[i-1, k]$  then

sırt çantasındaki  $i$  elemanını işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$  //  $i$  elemanı sırt çantasında değildir.



# Sırt çantasındaki elemanları bulmak

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 | 0 | 3 | 4 | 5 | 7 |

$i=4$

$k=5$

$b_i=6$

$w_i=5$

$V[i,k] = 7$

$V[i-1,k] = 7$

$i=n, k=W$

while  $i, k > 0$

if  $V[i,k] \neq V[i-1,k]$  then

sırt çantasındaki  $i$ 'nci elemanını işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$

# Sırt çantasındaki elemanları bulmak(2)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 | 0 | 3 | 4 | 5 | 7 |

$i=4$

$k=5$

$b_i=6$

$w_i=5$

$V[i,k] = 7$

$V[i-1,k] = 7$

$i=n, k=W$

while  $i, k > 0$

if  $V[i,k] \neq V[i-1,k]$  then

sırt çantasındaki  $i$ 'nci elemanı işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$

# Sırt çantasındaki elemanları bulmak(3)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 | 0 | 3 | 4 | 5 | 7 |

$i=3$

$k=5$

$b_i=5$

$w_i=4$

$V[i,k] = 7$

$V[i-1,k] = 7$

$i=n, k=W$

while  $i, k > 0$

if  $V[i,k] \neq V[i-1,k]$  then

sırt çantasındaki  $i$ 'nci elemanı işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$

# Sırt çantasındaki elemanları bulmak(4)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 | 0 | 3 | 4 | 5 | 7 |

$i=2$

$k=5$

$b_i=4$

$w_i=3$

$V[i,k] = 7$

$V[i-1,k] = 3$

$k - w_i = 2$

$i=n, k=W$

while  $i, k > 0$

if  $V[i,k] \neq V[i-1,k]$  then

sırt çantasındaki  $i$  elemanını işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$

# Sırt çantasındaki elemanları bulmak(5)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 | 0 | 3 | 4 | 5 | 7 |

$i=1$

$k=2$

$b_i=3$

$w_i=2$

$V[i,k] = 3$

$V[i-1,k] = 0$

$k - w_i = 0$

$i=n, k=W$

while  $i, k > 0$

if  $V[i,k] \neq V[i-1,k]$  then

sırt çantasındaki  $i$  elemanını işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$

# Sırt çantasındaki elemanları bulmak(6)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 | 0 | 3 | 4 | 5 | 7 |

$i=0$

$k=0$

Optimal Sırt çantası {1, 2} elemanlarını içerir.

$i=n, k=W$

while  $i, k > 0$

if  $V[i,k] \neq V[i-1,k]$  then

sırt çantasındaki  $i$  elemanını işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$

# Sırt çantasındaki elemanları bulmak(7)

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

| $i \backslash W$ | 0 | 1 | 2 | 3 | 4 | 5 |
|------------------|---|---|---|---|---|---|
| 0                | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                | 0 | 0 | 3 | 3 | 3 | 3 |
| 2                | 0 | 0 | 3 | 4 | 4 | 7 |
| 3                | 0 | 0 | 3 | 4 | 5 | 7 |
| 4                | 0 | 0 | 3 | 4 | 5 | 7 |

$i=n, k=W$

while  $i, k > 0$

if  $V[i, k] \neq V[i-1, k]$  then

sırt çantasındaki  $i$  elemanını işaretle

$i = i-1, k = k-w_i$

else

$i = i-1$

Optimal Sırt çantası {1, 2} elemanlarını içerir.

# 0-1 Knapsack Memory Kullanımı

```
for i = 1 to n      MFKnapsack(i, w)
    for w = 1 to W    if V[i,w] < 0
        V[i,w] = -1    if w < wi
                        value = MFKnapsack(i-1, w)
    for w = 0 to W    else
        V[0,w] = 0      value = max(MFKnapsack(i-1, w),
    for i = 1 to n      bi + MFKnapsack(i-1, w-wi))
        V[i,0] = 0      V[i,w] = value
                        return V[i,w]
```



# Brute-Force Yaklaşımı

```
public static int MCSS(int [] a) {  
  
    int max = 0, sum = 0, start = 0, end = 0;  
  
    // Cycle through all possible values of start and end indexes  
    // for the sum.  
    for (i = 0; i < a.length; i++) {  
        for (j = i; j < a.length; j++) {  
            sum = 0;  
  
            // Find sum A[i] to A[j].  
            for (k = i; k <= j; k++)  
                sum += a[k];  
            if (sum > max) {  
                max = sum;  
                start = i; // Although method doesn't return these  
                end = j; // they can be computed.  
            }  
        }  
    }  
    return max;  
}
```

# Dinamik Programlama Yaklaşımı

- (1)  $S_{Max}V(0) = 0$
- (2)  $MaxV(0) = 0$
- (3) for  $i=1$  to  $n$
- (4)  $S_{Max}V(i) = \max(S_{max}V(i-1)+x_i, 0)$
- (5)  $MaxV(i) = \max(MaxV(i-1), S_{Max}V(i))$
- (6) return  $MaxV(n)$
  
- Örnek Durum:
  - » 30, 40, -100, 10, 20, 50, -60, 90, -180, 100

# Açgözlü (Greedy) Yaklaşım

- Optimizasyon problemi (OP), bu konfigürasyonlarda tanımlanan objektif bir fonksiyonu minimize eden veya maksimuma çıkaran bir konfigürasyon bulmak için bir dizi konfigürasyonda arama yapılmasını içeren bir problemdir.
- Açgözlü yöntem, bir dizi (uygulanabilir) seçimlerden geçen belirli bir OP'yi çözer
- Çözüm en uygun başlangıç konfigürasyonundan başlar ve daha sonra o anda mümkün olanlardan en iyi görünen kararı tekrar tekrar verir.

# Açgözlü Yöntem(The Greedy Method)

- Açgözlü yaklaşım her zaman optimal bir çözüme yol açmaz.
- Açgözlü bir çözümü olan problemler, açgözlü seçim özelliğine sahip problem olarak adlandırılır.
- Açgözlü yaklaşım, yaklaşık bir çözüm üretmek için zor (çözülmesi zor) problemler bağlamında da kullanılır.

# Fractional Knapsack (Kesirli Sırt Çantası ) Problem

- $n$  elemanlı  $S$  kümesi için, fraksiyonel sırt çantası probleminde, her bir  $i$  öğesi pozitif bir  $b_i$  faydasına ve pozitif bir  $w_i$  ağırlık değerine sahiptir ve **belirli bir ağırlığı aşmayan** maksimum faydalı alt kümesini bulmak istiyoruz.
- Ayrıca, her bir elemanın keyfi kesirli kısımlarını almamıza izin verilir.

# Fractional Knapsack (Kesirli Sırt Çantası ) Problem

- Yani, her  $i$  öğesinin  $x_i$  miktarını alabiliriz, böylece

$$0 \leq x_i \leq w_i \text{ her } i \in S \text{ ve } \sum_{i \in S} x_i \leq W$$

- # Alınan öğelerin toplam faydası, hedef fonksiyon tarafından belirlenir.

$$\sum_{i \in S} b_i(x_i/w_i)$$

# Fractional Knapsack (Kesirli Sirt Çantası ) Problem

**Algorithm** FractionalKnapsack( $S, W$ ):

**Input:** Set  $S$  of items, such that each item  $i \in S$  has a positive benefit  $b_i$  and a positive weight  $w_i$ ; positive maximum total weight  $W$

**Output:** Amount  $x_i$  of each item  $i \in S$  that maximizes the total benefit while not exceeding the maximum total weight  $W$

**for** each item  $i \in S$  **do**

$x_i \leftarrow 0$

$v_i \leftarrow b_i/w_i$       {*value index* of item  $i$ }

$w \leftarrow 0$       {total weight}

**while**  $w < W$  **do**

remove from  $S$  an item  $i$  with highest value index      {greedy choice}

$a \leftarrow \min\{w_i, W - w\}$       {more than  $W - w$  causes a weight overflow}

$x_i \leftarrow a$

$w \leftarrow w + a$

# Fractional Knapsack (Kesirli Sırt Çantası ) Problem

- Çözümde,  $S$  öğelerini saklamak için yığın tabanlı bir PQ kullanıyoruz, burada her öğenin anahtarı onun değer endeksidir.
- PQ ile, en büyük değer endeksine sahip bir öğeyi kaldıran her açgözlü seçim  $O(\log n)$  süresini alır
- Kesirli sırt çantası algoritması  $O(n \log n)$  zaman maliyetine sahiptir.



# Fractional Knapsack (Kesirli Sırt Çantası) Problemi

- *Kesirli sırt çantası sorunu, açgözlü seçim özelliğini sağlar eder, bu nedenle*
- Teorem:  $n$  öğeli  $S$  kümesiyle bir kesirli sırt çantası problemi örneği göz önüne alındığında,  $S$ 'nin maksimum karlı bir alt kümesini (toplam ağırlığı  $W$  olan kesirli miktarlara da izin vererek)  $O(n \log n)$  zamanında oluşturabiliriz.

Bir optimizasyon problemi için açgözlü bir algoritma her zaman şu anda en iyi görünen seçimi yapar ve mevcut alt çözüme ekler.

Açgözlü algoritmalar her zaman en uygun çözümleri sunmaz, ancak yaptıklarında genellikle mevcut en basit ve en verimli algoritmalardır.

# Fractional Knapsack

Her bir öğe için ikili (0-1) seçimler yapmak yerine öğelerin kesirleri alınabilir.

Kesirli Sırt Çantası Sorunu açgözlü strateji ile çözülebilirken, 0 - 1 sorunu çözülemez.

Her nesne için birim başına değeri hesaplayın.

Açgözlü bir Stratejiye uyararak, mümkün olduğunca birim başına en yüksek değere sahip öğeyi alıyoruz.

Eğer bu eleman kalmazsa ve daha fazlasını taşıyabilmek mümkünse, bir sonraki en değerli elemandan mümkün olduğunca fazla değer alırız.

Sıralama, pound başına değere göre,  $O(n \log n)$  zamanında çalışan açgözlü algoritma

Fractional Knapsack (Array v, Array w, int W)

1. for i= 1 to size (v)
2. do p [i] = v [i] / w [i]
3. Sort-Descending (p)
4. i  $\leftarrow$  1
5. while (W>0)
6. do amount = min (W, w [i])
7. solution [i] = amount
8. W= W-amount
9. i  $\leftarrow$  i+1
10. return solution

$$I = (I_1, I_2, I_3, I_4, I_5)$$

$$w = (5, 10, 20, 30, 40)$$

$$v = (30, 20, 100, 90, 160)$$

The capacity of knapsack  $W = 60$

Şimdi sırt çantasını pi'nin azalan değerine göre doldurun.

İlk olarak, ağırlığı 5 olan  $I_1$  ögesini seçiyoruz.

Sonra ağırlığı 20 olan ürün  $I_3$ 'ü seçin. Sırt çantasının toplam ağırlığı  $20+5=25$

Şimdi bir sonraki öge  $I_5$  ve ağırlığı 40, ancak sadece 35 istiyoruz, bu yüzden onun kesirli kısmını seçtik,

$$5 \times \frac{5}{5} + 20 \times \frac{20}{20} + 40 \times \frac{35}{40}$$

$$\text{Weight} = 5 + 20 + 35 = 60$$

Maximum Value:-

$$30 \times \frac{5}{5} + 100 \times \frac{20}{20} + 160 \times \frac{35}{40}$$

$$= 30 + 100 + 140 = 270 \text{ (Minimum Cost)}$$

| ITEM  | $w_i$ | $v_i$ |
|-------|-------|-------|
| $I_1$ | 5     | 30    |
| $I_2$ | 10    | 20    |
| $I_3$ | 20    | 100   |
| $I_4$ | 30    | 90    |
| $I_5$ | 40    | 160   |

| ITEM  | $w_i$ | $v_i$ | $P_i = \frac{v_i}{w_i}$ |
|-------|-------|-------|-------------------------|
| $I_1$ | 5     | 30    | 6.0                     |
| $I_2$ | 10    | 20    | 2.0                     |
| $I_3$ | 20    | 100   | 5.0                     |
| $I_4$ | 30    | 90    | 3.0                     |
| $I_5$ | 40    | 160   | 4.0                     |

| ITEM  | $w_i$ | $v_i$ | $P_i = \frac{v_i}{w_i}$ |
|-------|-------|-------|-------------------------|
| $I_1$ | 5     | 30    | 6.0                     |
| $I_3$ | 20    | 100   | 5.0                     |
| $I_5$ | 40    | 160   | 4.0                     |
| $I_4$ | 30    | 90    | 3.0                     |
| $I_2$ | 10    | 20    | 2.0                     |

$$5 \times \frac{5}{5} + 20 \times \frac{20}{20} + 40 \times \frac{35}{40}$$

$$\text{Weight} = 5 + 20 + 35 = 60$$

**Maximum Value:-**

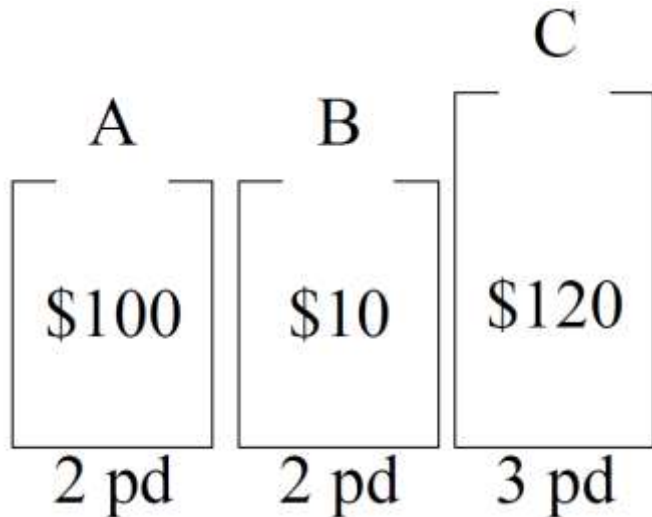
$$30 \times \frac{5}{5} + 100 \times \frac{20}{20} + 160 \times \frac{35}{40}$$

$$= 30 + 100 + 140 = 270 \text{ (Minimum Cost)}$$

## Örnek 2

Sırt çantası sorununu inceliyoruz ve kesirli sırt çantası için açgözlü bir algoritma görüyoruz. Ayrıca açgözlülüğün 0-1 sırt çantası için (DP kullanılarak çözülmesi gerekir) işe yaramadığını da görüyoruz.

Bir hırsız bir mağazaya girer ve aşağıdaki eşyaları görür:



Sırt Çantası 4 kilo.  
Kârı en üst düzeye çıkarmak için ne çalmalı?



Kesirli Sırt Çantası Sorunu: Hırsız bir öğenin bir kısmını alabilir.

Solution = + 

|                    |
|--------------------|
| 2 pounds of item A |
| 2 pounds of item C |

|                     |                    |
|---------------------|--------------------|
| 2 pds<br>A<br>\$100 | 2 pds<br>C<br>\$80 |
|---------------------|--------------------|

0-1 Sırt Çantası Sorunu: Hırsız yalnızca eşyayı alabilir veya bırakabilir. Kesir alamaz.

Solution =

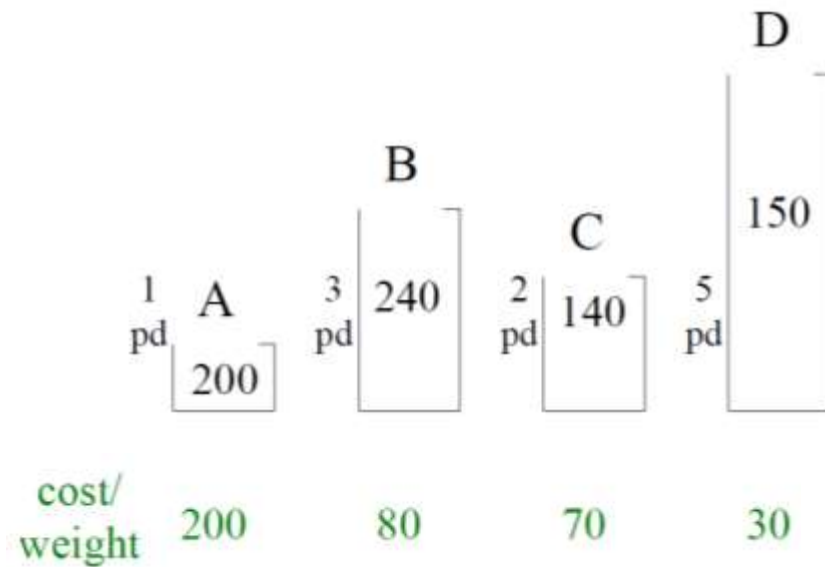
3 pounds of item C

|                     |  |
|---------------------|--|
| 3 pds<br>C<br>\$120 |  |
|---------------------|--|

# Greedy solution for Fractional Knapsack

Pound başına maliyeti büyükten küçüğe doğru sıralayın.

Sırt çantası kapasitesi 5 pds ise, çözüm



|   |     |   |
|---|-----|---|
| 1 | pds | A |
| 3 | pds | B |
| 1 | pds | C |

P, sonuçta ortaya çıkan maliyet (değer) maksimum olacak şekilde K ağırlık limiti ile öge seçme sorunu olsun.

|        |       |       |         |       |
|--------|-------|-------|---------|-------|
| weight | $w_1$ | $w_2$ | $\dots$ | $w_n$ |
| cost   | $c_1$ | $c_2$ | $\dots$ | $c_n$ |

1.  $i = 1, 2, \dots, n$ . için  $v_i = \frac{c_i}{w_i}$  değerlerini hesapla.
2. Hesaplanan bu değerleri azalan sırada sırala. Bu sıralı dizi elemanları  $(v_i), 1, 2, \dots, i, \dots, n$  ve ilişkili ağırlık değerleri ise  $w_i$  olsun.
3. Mevcut ağırlık limiti  $k$  olsun. ( $k=K$  başlangıçta), Her iterasyonda listenin başından  $i$  elemanını alacağız. Eğer  $k \geq w_i$  ise bu  $i$  elemanı  $k = k - w_i$  yaparız. Sonra seçilmemişler listesinden sıradaki elemanı alırız.

Eğer  $k < W_i$  ise  $i$ 'nin  $f$  kadar kesirini alırız. Bu durumda  $i$ 'nin sadece  $f = \frac{k}{w_i} (< 1)$  kısmını alırız. Ağırlık tam olarak  $k$  kapasitesine ulaşınca işlem tamamlanır.

$$K = 4$$

|                 |         |         |         |
|-----------------|---------|---------|---------|
|                 | A       | B       | C       |
|                 | 3<br>pd | 2<br>pd | 2<br>pd |
|                 | 300     | 190     | 180     |
| cost/<br>weight | 100     | 95      | 90      |

item B + item C